

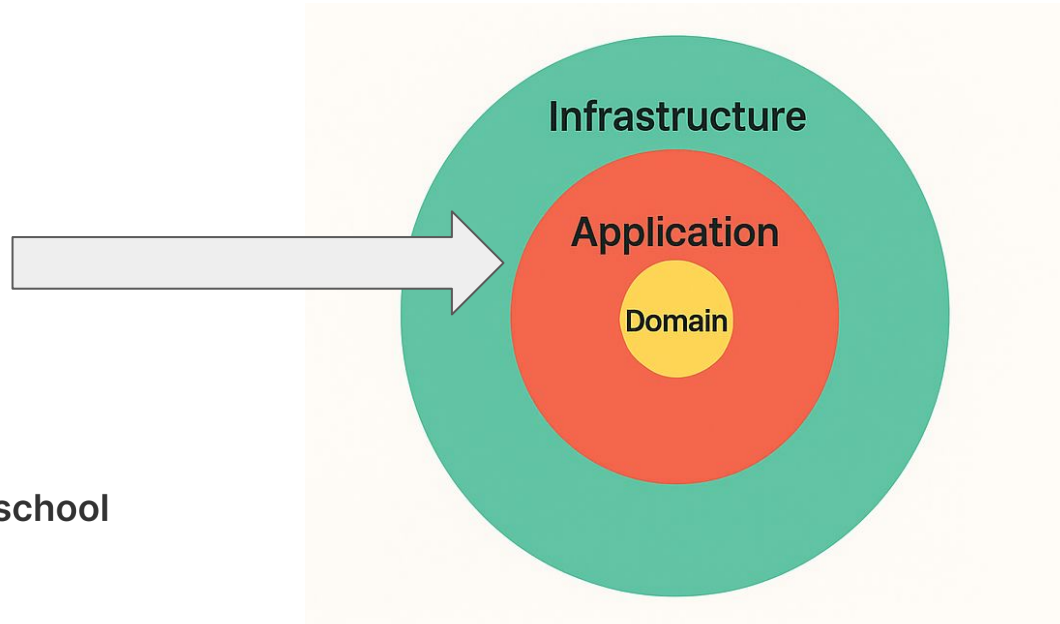
MÓDULO 2: ARQUITECTURA DE SOFTWARE

CASOS DE USO: LÓGICA DE LA APLICACIÓN

APLICANDO CLEAN ARCHITECTURE CON TYPESCRIPT

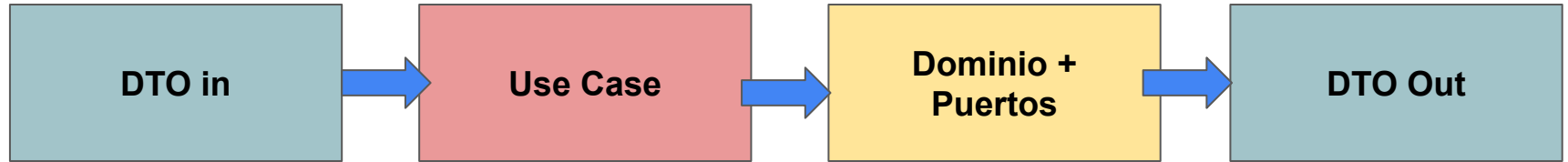
Casos de Uso: Lógica de la aplicación

Su rol: orquestación. No calculo fino



Papel en la capa de aplicación

Su rol: orquestación. No calculo fino

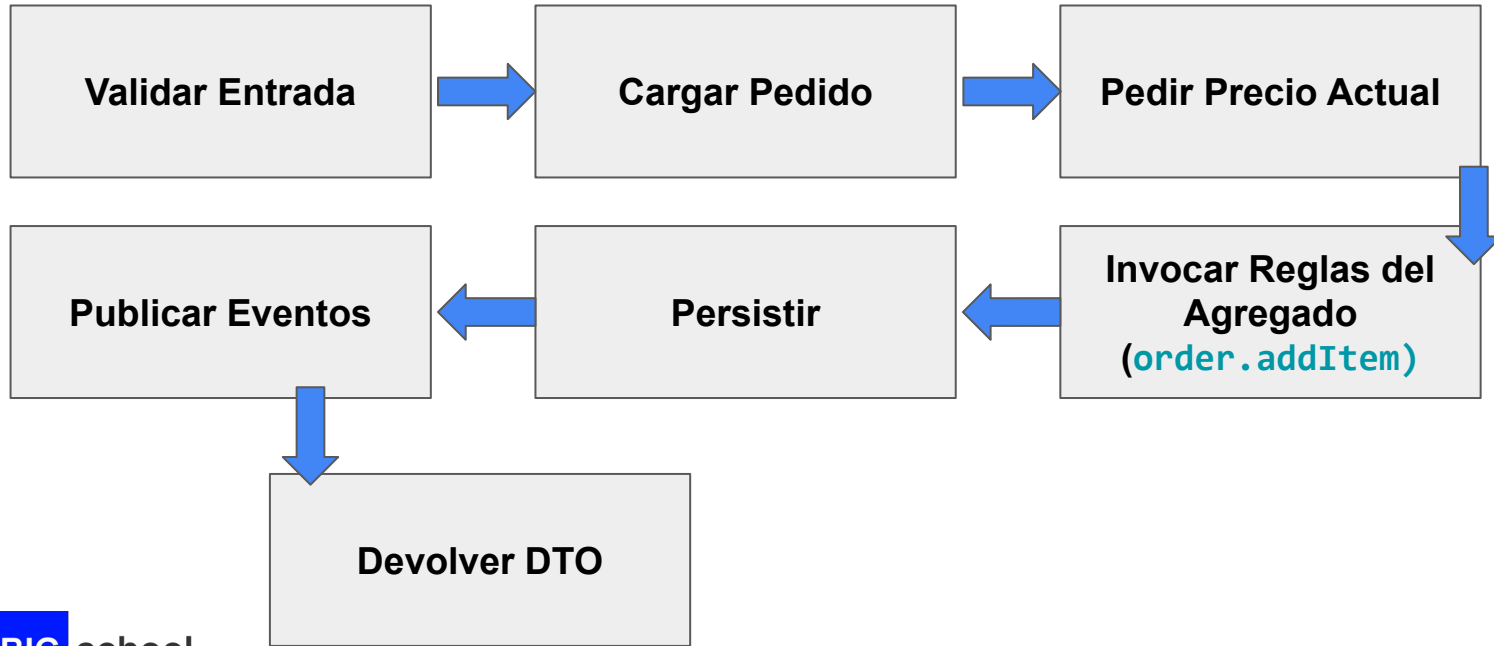


Ejemplo de patrón result

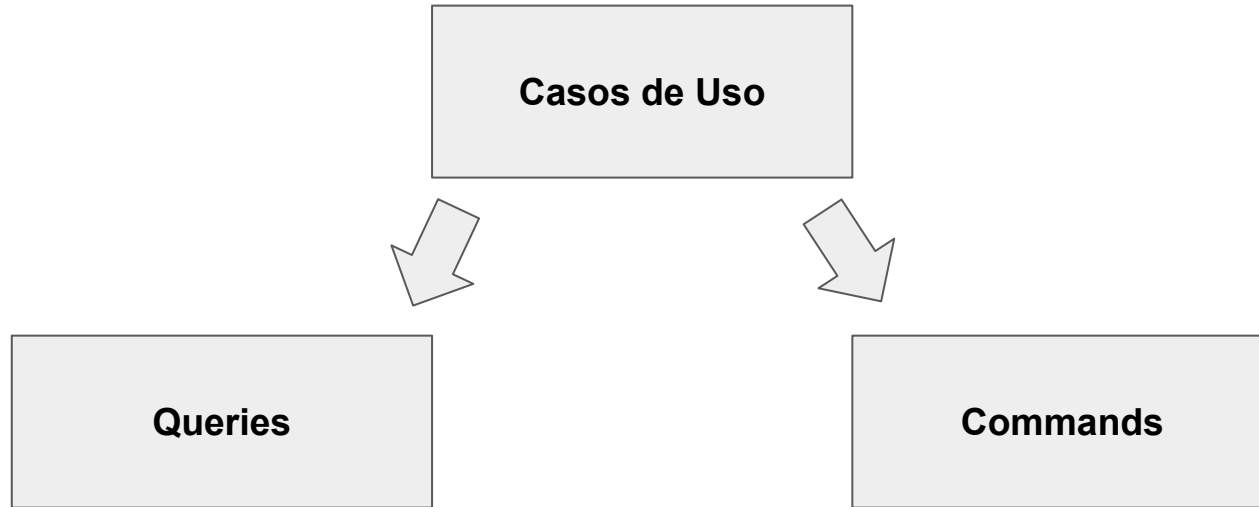
```
// src/shared/result.ts
export type Ok<T> = { ok: true; value: T }
export type Fail<E> = { ok: false; error: E }
export type Result<T, E> = Ok<T> | Fail<E>
export const ok = <T>(value: T): Ok<T> => ({ ok: true, value })
export const fail = <E>(error: E): Fail<E> => ({ ok: false, error })

// src/application/errors.ts
export type ValidationError = { type: "validation"; message: string;
  details?: Record<string, string> }
export type NotFoundError = { type: "not_found"; resource: string; id:
  string }
export type ConflictError = { type: "conflict"; message: string }
export type InfraError = { type: "infrastructure"; message: string }
export type AppError = ValidationError | NotFoundError | ConflictError |
  InfraError
```

Caso de Estudio: AddItemToOrder



CQRS



Antipatrones

Devolver **entidades** desde el caso de uso.

Acoplar a **frameworks** (usar `Request/Response`, `Prisma`, etc.).

Convertir el caso de uso en “**mini-controller**” con lógica de serialización.

Capturar **excepciones genéricas** y tragarlas.

No modelar **errores** como tipos (todo `throw new Error("...")`).



IA como Copiloto

Diseñar un caso de uso a partir de una historia

“Dada esta historia {texto}, propón DTOs in/out, puertos necesarios, Result types y un flujo paso a paso para el caso de uso. No uses librerías ni tipos del framework.”

Generar fakes de puertos para tests

“Crea dobles en memoria que implementen estas interfaces {OrderRepository, PricingService, EventBus} y ejemplos de uso en Vitest.”

Revisión de errores

“Analiza este caso de uso y clasifica cada `throw/fail` como `validation`, `not_found`, `conflict` o `infrastructure`. Sugiere mensajes y estructura uniforme.”

Mapeo entidad → DTO

“Dada la entidad `Order` y este DTO de salida, genera una función pura `toOrderSummaryDTO(order)` que no filtre detalles internos.”